

Part I

Policy-Based and Value-Based Algorithms

- Chapter 2** REINFORCE
- Chapter 3** SARSA
- Chapter 4** Deep Q-Networks (DQN)
- Chapter 5** Improving DQN

This page intentionally left blank

REINFORCE

This chapter introduces the first algorithm of the book, REINFORCE.

The REINFORCE algorithm, invented by Ronald J. Williams in 1992 in his paper “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning” [148], learns a parametrized policy which produces action probabilities from states. Agents use this policy directly to act in an environment.

The key idea is that during learning, actions that resulted in good outcomes should become more probable—these actions are positively *reinforced*. Conversely, actions which resulted in bad outcomes should become less probable. If learning is successful, over the course of many iterations action probabilities produced by the policy shift to distribution that results in good performance in an environment. Action probabilities are changed by following the *policy gradient*, therefore REINFORCE is known as a policy gradient algorithm.

The algorithm needs three components:

1. A parametrized policy
2. An objective to be maximized
3. A method for updating the policy parameters

Section 2.1 introduces parametrized policies. Then, in Section 2.2 we discuss the objective function which defines how to evaluate outcomes. Section 2.3 contains the core of the REINFORCE algorithm—the policy gradient. The policy gradient provides a way to estimate the gradient of the objective with respect to the policy parameters. This is a crucial step since the policy gradient is used to modify the policy parameters so as to maximize the objective.

After introducing the algorithm in Section 2.5, we discuss some limitations and introduce some strategies for improving performance. This is the subject of Section 2.5.1.

The chapter ends with two implementations of the algorithm. The first is minimal and self-contained. The second shows how it is implemented in SLM Lab.

2.1 Policy

A policy π is a function, mapping states to action probabilities, which is used to sample an action $a \sim \pi(s)$. In REINFORCE, an agent learns a policy and uses this to act in an environment.

A good policy is one which maximizes the cumulative discounted rewards. The key idea of the algorithm is to learn a good policy, and this means doing function approximation. Neural networks are powerful and flexible function approximators, so we can represent a policy using a deep neural network consisting of learnable parameters θ . This is often referred to as a policy network π_θ . We say that the policy is parametrized by θ .

Each specific set of values of the parameters of the policy network represents a particular policy. To see why, consider $\theta_1 \neq \theta_2$. For any given state s , different policy networks may output different sets of action probabilities, that is, $\pi_{\theta_1}(s) \neq \pi_{\theta_2}(s)$. The mappings from states to action probabilities are different so we say that π_{θ_1} and π_{θ_2} are different policies. A single neural network is therefore capable of representing many different policies.

Formulated in this way, the process of learning a good policy corresponds to searching for a good set of values for θ . For this reason, it is important that the policy network is differentiable. We will see in Section 2.3 that the mechanism by which the policy is improved is through gradient ascent in parameter space.

2.2 The Objective Function

In this section, we define the objective that is maximized by an agent in the REINFORCE algorithm. An objective can be understood as an agent's goal, such as winning a game or getting the highest score possible. First, we introduce the concept of a *return*, which is calculated using a trajectory. We then use this to formulate the objective.

Recall from Chapter 1 that an agent acting in an environment generates a trajectory, which contains a sequence of rewards along with the states and actions. A trajectory is denoted $\tau = s_0, a_0, r_0, \dots, s_T, a_T, r_T$.

The *return* of a trajectory $R_t(\tau)$ is defined as a discounted sum of rewards from time step t to the end of a trajectory, as shown in Equation 2.1.

$$R_t(\tau) = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} \quad (2.1)$$

Note that the sum starts from time step t , but the power that the discount factor γ is raised to, when summing for the return, starts from 0, so we need to offset the power by the starting time step t using $t' - t$.

When $t = 0$, the return is simply the return of the complete trajectory. This is also written as $R_0(\tau) = R(\tau)$ for brevity. The *objective* is the expected return over all complete trajectories generated by an agent. This is defined in Equation 2.2.

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (2.2)$$

Equation 2.2 says that the expectation is calculated over many trajectories sampled from a policy, that is, $\tau \sim \pi_\theta$. This expectation approaches the true value as more samples are gathered, and it is tied to the specific policy π_θ used.

2.3 The Policy Gradient

We have now defined the policy π_θ and the objective $J(\pi_\theta)$ which are two crucial ingredients for deriving the policy gradient algorithm. The policy provides a way for an agent to act, and the objective provides a target to maximize.

The final component of the algorithm is the policy gradient. Formally, we say a policy gradient algorithm solves the following problem:

$$\max_{\theta} J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \quad (2.3)$$

To maximize the objective, we perform gradient ascent on the policy parameters θ . Recall from calculus that the gradient points in the direction of steepest ascent. To improve on the objective,¹ compute the gradient and use it to update the parameters as shown in Equation 2.4.²

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\pi_\theta) \quad (2.4)$$

α is a scalar, known as the learning rate, which controls the size of the parameter update. The term $\nabla_{\theta} J(\pi_\theta)$ is known as the *policy gradient*. It is defined in Equation 2.5.

$$\nabla_{\theta} J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T R_t(\tau) \nabla_{\theta} \log \pi_\theta(a_t | s_t) \right] \quad (2.5)$$

The term $\pi_\theta(a_t | s_t)$ is the probability of the action taken by the agent at time step t . The action is sampled from the policy, $a_t \sim \pi_\theta(s_t)$. The right-hand side of the equation states that the gradient of the log probability of the action with respect to θ is multiplied by return $R_t(\tau)$.

Equation 2.5 states that the gradient of the objective is equivalent to the expected sum of the gradients of the log probabilities of the actions a_t multiplied by the corresponding returns $R_t(\tau)$. The full derivation is shown in Section 2.3.1.

The policy gradient is the mechanism by which action probabilities produced by the policy are changed. If the return $R_t(\tau) > 0$, then the probability of the action $\pi_\theta(a_t | s_t)$ is increased; conversely, if the return $R_t(\tau) < 0$, then the probability of the action

1. The objective $J(\pi_\theta)$ can be thought of as an abstract hypersurface on which we try to find the maximum point with θ as the variables. A hypersurface is a generalization of an ordinary 2D surface residing in 3D space into higher dimensions. A hypersurface residing in N -dimensional space is an object with $(N - 1)$ dimensions.
 2. Note that the parameter update can be done with any suitable optimizer that takes a $\nabla_{\theta} J(\pi_\theta)$ as input.

$\pi_\theta(a_t | s_t)$ is decreased. Over the course of many updates (Equation 2.4), the policy will learn to produce actions which result in high $R_t(\tau)$.

Equation 2.5 is the foundation of all policy gradient methods. REINFORCE was the first algorithm to use it in its simplest form. Newer algorithms build on top of this by modifying the function to improve performance, as we will see in Chapters 6 and 7. However, a final question remains—how does one implement and estimate the ideal policy gradient equation?

2.3.1 Policy Gradient Derivation

Here we derive the policy gradient (Equation 2.5) from the gradient of the objective shown in Equation 2.6. Note that this section can be skipped on a first reading.

$$\nabla_\theta J(\pi_\theta) = \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \quad (2.6)$$

Equation 2.6 presents a problem because we cannot differentiate $R(\tau) = \sum_{t=0}^T \gamma^t r_t$ with respect to θ .³ The rewards r_t are generated by an unknown reward function $\mathcal{R}(s_t, a_t, s_{t+1})$ which cannot be differentiated. The only way for the policy variables θ to influence $R(\tau)$ is by changing the state and action distributions which, in turn, change the rewards received by an agent.

We therefore need to transform Equation 2.6 into a form where we can take a gradient with respect to θ . To do so, we'll use some handy identities.

Given a function $f(x)$, a parametrized probability distribution $p(x | \theta)$, and its expectation $\mathbb{E}_{x \sim p(x|\theta)}[f(x)]$, the gradient of the expectation can be rewritten as follows:

$$\begin{aligned} \nabla_\theta \mathbb{E}_{x \sim p(x|\theta)}[f(x)] &= \nabla_\theta \int dx f(x) p(x | \theta) && (\text{definition of expectation}) \end{aligned} \quad (2.7)$$

$$= \int dx \nabla_\theta (p(x | \theta) f(x)) \quad (\text{bring in } \nabla_\theta) \quad (2.8)$$

$$= \int dx (f(x) \nabla_\theta p(x | \theta) + p(x | \theta) \nabla_\theta f(x)) \quad (\text{chain-rule}) \quad (2.9)$$

$$= \int dx f(x) \nabla_\theta p(x | \theta) \quad (\nabla_\theta f(x) = 0) \quad (2.10)$$

$$= \int dx f(x) p(x | \theta) \frac{\nabla_\theta p(x | \theta)}{p(x | \theta)} \quad \left(\text{multiply } \frac{p(x | \theta)}{p(x | \theta)} \right) \quad (2.11)$$

$$= \int dx f(x) p(x | \theta) \nabla_\theta \log p(x | \theta) \quad (\text{substitute Equation 2.14}) \quad (2.12)$$

$$= \mathbb{E}_x[f(x) \nabla_\theta \log p(x | \theta)] \quad (\text{definition of expectation}) \quad (2.13)$$

3. However, the gradient can be estimated using black-box optimization techniques, such as Augmented Random Search [83]. This is beyond the scope of this book.

This identity says that the gradient of an expectation is equivalent to the expectation of the gradient of log probability multiplied by the original function. The first line is simply the definition of an expectation. An integral form is used for generality by considering $f(x)$ to be a continuous function, but it equally applies to a summation form for a discrete function. The next three lines follow from basic calculus.

Notice that Equation 2.10 has solved our initial problem since we can take the gradient of $p(x | \theta)$, but $f(x)$ is a black-box function which cannot be integrated. To deal with this, we need to convert the equation into an expectation so that it can be estimated through sampling. First, multiply it identically by $\frac{p(x|\theta)}{p(x|\theta)}$ in Equation 2.11. The resulting fraction $\frac{\nabla_{\theta} p(x|\theta)}{p(x|\theta)}$ can be rewritten with the *log-derivative trick* in Equation 2.14.

$$\nabla_{\theta} \log p(x | \theta) = \frac{\nabla_{\theta} p(x | \theta)}{p(x | \theta)} \quad (2.14)$$

Substituting Equation 2.14 into 2.11 gives Equation 2.12. This can be written as an expectation to give Equation 2.13. Finally, we simply rewrite the expression as an expectation.

Now, it should be apparent that this identity can be applied to our objective. By substituting $x = \tau$, $f(x) = R(\tau)$, $p(x | \theta) = p(\tau | \theta)$, Equation 2.6 can be written as

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) \nabla_{\theta} \log p(\tau | \theta)] \quad (2.15)$$

However, the term $p(\tau | \theta)$ in Equation 2.15 needs to relate to the policy π_{θ} , which we have control over. Therefore, it needs to be expanded further.

Observe that the trajectory τ is just a sequence of interleaved events, a_t and s_{t+1} , sampled, respectively, from the agent's action probability $\pi_{\theta}(a_t | s_t)$ and the environment's transition probability $p(s_{t+1} | s_t, a_t)$. Since the probabilities are conditionally independent, the probability of the entire trajectory is the product of the individual probabilities, as shown in Equation 2.16.

$$p(\tau | \theta) = \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t) \quad (2.16)$$

Apply logarithms⁴ to both sides to match Equation 2.16 with Equation 2.15.

$$\log p(\tau | \theta) = \log \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t) \quad (2.17)$$

$$\log p(\tau | \theta) = \sum_{t \geq 0} (\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t)) \quad (2.18)$$

4. One benefit from using log probabilities is that they are numerically stabler than plain probabilities. Certain operations also become faster to compute. As a quick exercise, try to figure out the reasons for both these benefits.

$$\nabla_{\theta} \log p(\tau | \theta) = \nabla_{\theta} \sum_{t \geq 0} (\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t)) \quad (2.19)$$

$$\nabla_{\theta} \log p(\tau | \theta) = \nabla_{\theta} \sum_{t \geq 0} \log \pi_{\theta}(a_t | s_t) \quad (2.20)$$

Equation 2.18 follows from the fact that logarithm of product equals the sum of logarithms of its components. From there, we can apply the gradient ∇_{θ} to both sides to get Equation 2.19. The gradient can be moved inside the summation terms. Since $\log p(s_{t+1} | s_t, a_t)$ is independent of θ , its gradient is zero, and it can be dropped. This yields Equation 2.20 that expresses the probability $p(\tau | \theta)$ in terms of $\pi_{\theta}(a_t | s_t)$. Also, note that the trajectory τ on the left corresponds to a summation of its individual time steps t on the right.

With this, we are finally ready to rewrite $\nabla_{\theta} J(\pi_{\theta})$ from Equation 2.6 in a form that can be differentiated. By substituting Equation 2.20 into 2.15 and bringing in the multiplier $R(\tau)$, we obtain

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T R_t(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (2.21)$$

Our problem was that Equation 2.6 contained a function that was not differentiable. After a series of transformations, we arrived at Equation 2.20. This can be estimated quite easily using a policy network π_{θ} , with the gradient calculation handled by the automatic differentiation feature available in neural network libraries.

2.4 Monte Carlo Sampling

The REINFORCE algorithm numerically estimates the policy gradient using Monte Carlo sampling.

Monte Carlo sampling refers to any method that uses random sampling to generate data used to approximate a function. In essence, it is just “approximation with random sampling.” It is a technique which became popular thanks to Stanislaw Ulam, a mathematician who worked at the Los Alamos research lab in the 1940s.

To see how Monte Carlo works, let’s work through an example of how it can be used to estimate the value of π (the mathematical constant)—the ratio of a circle’s circumference to its diameter.⁵ A Monte Carlo approach to solving this problem is to take a circle of radius $r = 1$ centered at the origin and inscribe it in a square. Their areas are πr^2 and $(2r)^2$, respectively. Hence, the ratio of these areas is simply

$$\frac{\text{area of circle}}{\text{area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4} \quad (2.22)$$

5. Just for this example, we stop using π for policy, but that usage will resume after this.

Numerically, the square has an area of 4, but since we do not yet know π , so the area of the circle is unknown. Let's consider one quadrant. To obtain an estimation for π , sample many points within the square using a uniformly random distribution. A point (x, y) that lands in the circle has distance less than 1 from the origin—that is, $\sqrt{(x-0)^2 + (y-0)^2} \leq 1$. This is shown in Figure 2.1. Using this, if we count the number of points in the circle, then count the number of points sampled in total, their ratio roughly equals to Equation 2.22. By iteratively sampling more points and updating the ratio, our estimation will get closer to the precise value. Multiplying this ratio by 4 gives us the estimated value of $\pi \approx 3.14159$.

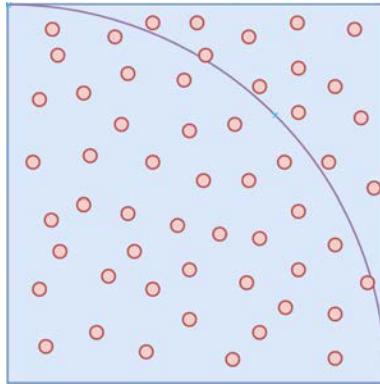


Figure 2.1 Monte Carlo sampling used to estimate π

Now, let's return to deep RL and look at how Monte Carlo can be used to numerically estimate the policy gradient in Equation 2.5. It is very straightforward. The expectation $\mathbb{E}_{\tau \sim \pi_\theta}$ implies that as more trajectories τ s are sampled using a policy π_θ and averaged, it approaches the actual policy gradient $\nabla_\theta J(\pi_\theta)$. Instead of sampling many trajectories per policy, we can sample just one as shown in Equation 2.23.

$$\nabla_\theta J(\pi_\theta) \approx \sum_{t=0}^T R_t(\tau) \nabla_\theta \log \pi_\theta(a_t | s_t) \quad (2.23)$$

This is how policy gradient is implemented—as a Monte Carlo estimate over sampled trajectories.

With all the components now in place, let's look at the REINFORCE algorithm.

2.5 REINFORCE Algorithm

This section discusses the REINFORCE algorithm and introduces the concept of an on-policy algorithm. We then look at some of its limitations and introduce a baseline to improve performance.

The algorithm is shown in Algorithm 2.1. It is very simple. First, initialize the learning rate α and construct a policy network π_θ with randomly initialized weights.

Next, iterate for multiple episodes as follows: use the policy network π_θ to generate a trajectory $\tau = s_0, a_0, r_0, \dots, s_T, a_T, r_T$ for an episode. Then, for each time step t in the trajectory, compute the return $R_t(\tau)$. Use $R_t(\tau)$ to estimate the policy gradient. Sum the policy gradients for all time steps, then use the result to update the policy network parameters θ .

Algorithm 2.1 REINFORCE algorithm

```

1: Initialize learning rate  $\alpha$ 
2: Initialize weights  $\theta$  of a policy network  $\pi_\theta$ 
3: for  $episode = 0, \dots, MAX\_EPISODE$  do
4:   Sample a trajectory  $\tau = s_0, a_0, r_0, \dots, s_T, a_T, r_T$ 
5:   Set  $\nabla_\theta J(\pi_\theta) = 0$ 
6:   for  $t = 0, \dots, T$  do
7:      $R_t(\tau) = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ 
8:      $\nabla_\theta J(\pi_\theta) = \nabla_\theta J(\pi_\theta) + R_t(\tau) \nabla_\theta \log \pi_\theta(a_t | s_t)$ 
9:   end for
10:   $\theta = \theta + \alpha \nabla_\theta J(\pi_\theta)$ 
11: end for

```

It is important that a trajectory is discarded after each parameter update—it cannot be reused. This is because REINFORCE is an *on-policy* algorithm. Recall from Section 1.4 that an algorithm is on-policy if the parameter update equation depends on the current policy. This is clear from line 8 since the policy gradient directly depends on action probabilities $\pi_\theta(a_t | s_t)$ generated by the current policy π_θ , but not some past policy $\pi_{\theta'}$. Correspondingly, the return $R_t(\tau)$ where $\tau \sim \pi_\theta$ must also be generated from π_θ , otherwise the action probabilities will be adjusted based on returns that the policy wouldn't have generated.

2.5.1 Improving REINFORCE

Our formulation of the REINFORCE algorithm estimates the policy gradient using Monte Carlo sampling with a single trajectory. This is an unbiased estimate of the policy gradient, but one disadvantage of this approach is that it has a high variance. In this section, we introduce a baseline to reduce the variance of the estimate. Following this, we will also discuss reward normalization to address the issue of reward scaling.

When using Monte Carlo sampling, the policy gradient estimate may have high variance because the returns can vary significantly from trajectory to trajectory. This is due to three factors. First, actions have some randomness because they are sampled from a probability distribution. Second, the starting state may vary per episode. Third, the environment transition function may be stochastic.

One way to reduce the variance of the estimate is to modify the returns by subtracting a suitable action-independent baseline, as shown in Equation 2.24.

$$\nabla_{\theta} J(\pi_{\theta}) \approx \sum_{t=0}^T (R_t(\tau) - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \quad (2.24)$$

One option for the baseline is the value function V^{π} . This choice of baseline motivates the Actor-Critic algorithm. We leave the discussion of this to Chapter 6.

An alternative is to use the mean returns over the trajectory. Let $b = \frac{1}{T} \sum_{t=0}^T R_t(\tau)$. Note that this is a constant baseline per trajectory that does not vary with state s_t . It has the effect of centering the returns for each trajectory around 0. For each trajectory, on average, the best 50% of the actions will be encouraged, and the others discouraged.

To see why this is useful, consider the case where all the rewards for an environment are negative. Without a baseline, even when an agent produces a very good action, it gets discouraged because the returns are always negative. Over time, this can still result in good policies since worse actions will get discouraged even more, thus indirectly increasing the probabilities of better actions. However, it can lead to slower learning because probability adjustments can only be made in one direction. The converse happens for environments where all the rewards are positive. Learning is more effective when we can both increase and decrease the action probabilities. This requires having both positive and negative returns.

2.6 Implementing REINFORCE

In this section we present two implementations. The first is a minimal, self-contained implementation. The second shows how the algorithm is implemented in SLM Lab; this is a slightly more advanced version that is integrated with the lab components.

2.6.1 A Minimal REINFORCE Implementation

A minimal implementation is useful as it closely corresponds to Algorithm 2.1. This is good practice for translating theory into code. REINFORCE is a good candidate for this because it is the simplest RL algorithm which can easily be implemented in a few lines of code (see Code 2.1).

Other RL algorithms are more complex and so benefit from being implemented within a common framework with reusable modular components. For this reason, to be consistent with the remaining algorithms in this book, we also present an implementation in SLM Lab. This serves as an introduction to the lab's algorithm API.

Code 2.1 A standalone working implementation of REINFORCE to solve CartPo1e-v0

```
1 from torch.distributions import Categorical
2 import gym
```

```

3  import numpy as np
4  import torch
5  import torch.nn as nn
6  import torch.optim as optim
7
8  gamma = 0.99
9
10 class Pi(nn.Module):
11     def __init__(self, in_dim, out_dim):
12         super(Pi, self).__init__()
13         layers = [
14             nn.Linear(in_dim, 64),
15             nn.ReLU(),
16             nn.Linear(64, out_dim),
17         ]
18         self.model = nn.Sequential(*layers)
19         self.onpolicy_reset()
20         self.train() # set training mode
21
22     def onpolicy_reset(self):
23         self.log_probs = []
24         self.rewards = []
25
26     def forward(self, x):
27         pdparam = self.model(x)
28         return pdparam
29
30     def act(self, state):
31         x = torch.from_numpy(state.astype(np.float32)) # to tensor
32         pdparam = self.forward(x) # forward pass
33         pd = Categorical(logits=pdparam) # probability distribution
34         action = pd.sample() # pi(a|s) in action via pd
35         log_prob = pd.log_prob(action) # log_prob of pi(a|s)
36         self.log_probs.append(log_prob) # store for training
37         return action.item()
38
39 def train(pi, optimizer):
40     # Inner gradient-ascent loop of REINFORCE algorithm
41     T = len(pi.rewards)
42     rets = np.empty(T, dtype=np.float32) # the returns
43     future_ret = 0.0
44     # compute the returns efficiently
45     for t in reversed(range(T)):
46         future_ret = pi.rewards[t] + gamma * future_ret

```

```

47         rets[t] = future_ret
48     rets = torch.tensor(rets)
49     log_probs = torch.stack(pi.log_probs)
50     loss = - log_probs * rets # gradient term; Negative for maximizing
51     loss = torch.sum(loss)
52     optimizer.zero_grad()
53     loss.backward() # backpropagate, compute gradients
54     optimizer.step() # gradient-ascent, update the weights
55     return loss
56
57 def main():
58     env = gym.make('CartPole-v0')
59     in_dim = env.observation_space.shape[0] # 4
60     out_dim = env.action_space.n # 2
61     pi = Pi(in_dim, out_dim) # policy pi_theta for REINFORCE
62     optimizer = optim.Adam(pi.parameters(), lr=0.01)
63     for epi in range(300):
64         state = env.reset()
65         for t in range(200): # cartpole max timestep is 200
66             action = pi.act(state)
67             state, reward, done, _ = env.step(action)
68             pi.rewards.append(reward)
69             env.render()
70             if done:
71                 break
72         loss = train(pi, optimizer) # train per episode
73         total_reward = sum(pi.rewards)
74         solved = total_reward > 195.0
75         pi.onpolicy_reset() # onpolicy: clear memory after training
76         print(f'Episode {epi}, loss: {loss}, \
77               total_reward: {total_reward}, solved: {solved}')
78
79 if __name__ == '__main__':
80     main()

```

Let's walk through the minimal implementation.

1. `Pi` constructs the policy network that is a simple one-layer MLP with 64 hidden units (lines 10–20).
2. `act` defines the method to produce action (lines 30–37).
3. `train` implements the update steps in Algorithm 2.1. Note that the loss is expressed as the sum of the negative log probabilities multiplied by the returns (lines 50–51). The negative sign is necessary because by default, PyTorch's optimizer minimizes the loss, whereas we want to maximize the objective. Furthermore, we formulate the

loss in this way to utilize PyTorch’s automatic differentiation feature. When we call `loss.backward()`, this computes the gradient of the loss (line 53), which is equal to the policy gradient. Finally, we update the policy parameters by calling `optimizer.step()` (line 54).

4. `main` is the main loop. It constructs a `CartPole` environment, the policy network `Pi`, and an optimizer. Then, it runs the training loop for 300 episodes. As training progresses, the total reward per episode should increase towards 200. The environment is solved when the total reward is above 195.

2.6.2 Constructing Policies with PyTorch

The implementation of a policy π_θ deserves a closer look. In this section, we will show how the outputs from a neural network are transformed into an action probability distribution that is used to sample an action, $a \sim \pi_\theta(s)$.

The key idea is that probability distributions can be parametrized, either by enumerating the full probabilities for a discrete distribution or by specifying the mean and standard deviation⁶ of a continuous distribution such as the normal distribution. These probability distribution parameters can be learned and output by a neural network.

To output an action, we first use a policy network to compute the probability distribution parameters from a state. Then, we use these parameters to construct an action probability distribution. Finally, we use the action probability distribution to sample an action and compute the action log probability.

Let’s see how this works in pseudocode. Algorithm 2.2 constructs a discrete-action probability distribution and uses it to compute the log probability. It constructs a categorical (multinomial) distribution, although other discrete distributions may be used. Since the network output is not necessarily normalized, the distribution parameters are treated as logits instead of probabilities. Algorithm 2.2 also closely corresponds to the PyTorch implementation of the policy in Code 2.1 (lines 31–35).

Algorithm 2.2 Constructing a discrete policy

- 1: Given a policy network `net`, a `Categorical` distribution class, and a `state`
 - 2: Compute the output `pdparams = net(state)`
 - 3: Construct an instance of an action probability distribution
 $\hookrightarrow$ `pd = Categorical(logits=pdparams)`
 - 4: Use `pd` to sample an action, `action = pd.sample()`
 - 5: Use `pd` and `action` to compute the action log probability,
 $\hookrightarrow$ `log_prob = pd.log_prob(action)`
-

A simple implementation of Algorithm 2.2 using PyTorch is shown in Code 2.2 with closely corresponding steps. For simplicity, we use a dummy policy network output. Inside

6. Some probability distributions require special parameters, but the same idea still applies.

the `Categorical` constructor, PyTorch will transform specified logits to probabilities when constructing a categorical distribution.

Code 2.2 A discrete policy implemented using categorical distribution

```

1 from torch.distributions import Categorical
2 import torch
3
4 # suppose for 2 actions (CartPole: move left, move right)
5 # we obtain their logit probabilities from a policy network
6 policy_net_output = torch.tensor([-1.6094, -0.2231])
7 # the pdparams are logits, equivalent to probs = [0.2, 0.8]
8 pdparams = policy_net_output
9 pd = Categorical(logits=pdparams)
10
11 # sample an action
12 action = pd.sample()
13 # => tensor(1), or 'move right'
14
15 # compute the action log probability
16 pd.log_prob(action)
17 # => tensor(-0.2231), log probability of 'move right'

```

Similarly, a continuous-action policy is constructed the same way. Algorithm 2.3 shows an example using a normal distribution. The normal distribution, like many other continuous distributions, is parametrized by a mean and standard deviation. In many scientific computing libraries, they are known, respectively, as `loc` and `scale` in code, to imply adjusting the location and scale of a distribution. Once the action probability distribution is constructed, the remaining steps for computing the action log probability are the same as in Algorithm 2.2.

Algorithm 2.3 Constructing a continuous policy

- 1: Given a policy network `net`, a `Normal` distribution class, and a *state*
 - 2: Compute the output `pdparams = net(state)`
 - 3: Construct an instance of an action probability distribution
 $\hookrightarrow$ `pd = Normal(loc=pdparams[0], scale=pdparams[1])`
 - 4: Use `pd` to sample an action, `action = pd.sample()`
 - 5: Use `pd` and `action` to compute the action log probability,
 $\hookrightarrow$ `log_prob = pd.log_prob(action)`
-

For completeness, Algorithm 2.3 is also implemented with PyTorch in Code 2.3.

Code 2.3 A continuous policy implemented using normal distribution

```
1 from torch.distributions import Normal
2 import torch
3
4 # suppose for 1 action (Pendulum: torque)
5 # we obtain its mean and std from a policy network
6 policy_net_output = torch.tensor([1.0, 0.2])
7 # the pdparams are (mean, std), or (loc, scale)
8 pdparams = policy_net_output
9 pd = Normal(loc=pdparams[0], scale=pdparams[1])
10
11 # sample an action
12 action = pd.sample()
13 # => tensor(1.0295), the amount of torque
14
15 # compute the action log probability
16 pd.log_prob(action)
17 # => tensor(0.6796), log probability of this torque
```

The generality of the policy construction workflow implies that it can be applied easily to both discrete and continuous-action environments. Its simplicity is also one of the strengths of a policy-based algorithm.

SLM Lab implements Algorithms 2.2 and 2.3 generally, for all policy-based algorithms, in a modular and reusable manner. Additionally, these implementations are also optimized for efficiency. For instance, the action log probabilities are computed all at once and only during training to speed up computation.

Having seen how REINFORCE can be implemented very simply, we now discuss the REINFORCE implementation in SLM Lab integrated with its components. This serves as an introduction to algorithm implementation using the SLM Lab framework. This REINFORCE code is used as the parent class which we will extend to implement more complex algorithms later in this book. As we use SLM Lab to run all the experiments discussed later, it's time to start becoming familiar with the framework.

In what follows, we discuss only the algorithm-specific methods and leave their integration into the lab framework for hands-on code exploration. The full code along with all the components required to run an RL training session can be found in the companion SLM Lab code repository.

2.6.3 Sampling Actions

Let's go through the action sampling methods of the `Reinforce` class shown in Code 2.4. Note that throughout this book, for clarity of exposition, some lines of code that are not essential to explaining the functionality of a method are omitted and replaced with

"...". Also note that methods tagged with `@lab_api` are standard API methods of an algorithm in SLM Lab.

`calc_pdparam` calculates the parameters for the action distribution. `calc_pdparam` is called by `self.action_policy` (line 15) which produces an action (this is described in more detail in Section 2.6.2). `act` samples an action from the algorithm's action policy, which can be a discrete or continuous distribution, for example `Categorical` and `Normal`.

Code 2.4 REINFORCE implementation: sampling actions from the policy network

```

1  # slm_lab/agent/algorithm/reinforce.py
2
3  class Reinforce(Algorithm):
4      ...
5
6      @lab_api
7      def calc_pdparam(self, x, net=None):
8          net = self.net if net is None else net
9          pdparam = net(x)
10         return pdparam
11
12     @lab_api
13     def act(self, state):
14         body = self.body
15         action = self.action_policy(state, self, body)
16         return action.cpu().squeeze().numpy() # squeeze to handle scalar

```

2.6.4 Calculating Policy Loss

Code 2.5 shows the methods for calculating the policy loss. There are two components. First, we need to calculate the reinforcing signal. This is handled by the method `calc_ret_adv` which calculates the returns for each element in the batch. Optionally, we subtract the baseline from the returns. In this example, it is simply the mean returns over the batch.

Once we have the returns, we can calculate the policy loss. In `calc_policy_loss`, we first obtain the action probability distribution (line 15) needed to calculate the action log probabilities (line 18). Next, we combine the returns (denoted `adv`s) with the log probabilities to form the policy loss (line 19). This has the same form as seen in Code 2.1 (lines 50–51), so that we can utilize PyTorch's automatic differentiation feature.

An optional addition to the loss is an entropy term to encourage exploration (lines 20–23). This is discussed in more detail in Section 6.3.

Code 2.5 REINFORCE implementation: calculating the policy loss

```

1  # slm_lab/agent/algorithm/reinforce.py
2
3  class Reinforce(Algorithm):
4      ...
5
6      def calc_ret_advds(self, batch):
7          rets = math_util.calc_returns(batch['rewards'], batch['dones'],
8          ↪ self.gamma)
9          if self.center_return:
10             rets = math_util.center_mean(rets)
11         advs = rets
12         ...
13         return advs
14
15     def calc_policy_loss(self, batch, pdparams, advs):
16         action_pd = policy_util.init_action_pd(self.body.ActionPD, pdparams)
17         actions = batch['actions']
18         ...
19         log_probs = action_pd.log_prob(actions)
20         policy_loss = - self.policy_loss_coef * (log_probs * advs).mean()
21         if self.entropy_coef_spec:
22             entropy = action_pd.entropy().mean()
23             self.body.mean_entropy = entropy # update logging variable
24             policy_loss += (-self.body.entropy_coef * entropy)
25         return policy_loss

```

2.6.5 REINFORCE Training Loop

Code 2.6 shows the training loop and associated memory sampling method. `train` makes one parameter update to the policy network using a batch of collected trajectories. Training will only be triggered once sufficient data has been collected. The trajectories are obtained from the agent's memory by calling `sample` (line 17).

After sampling a batch of data, we compute the action probability distribution parameters `pdparams` and the returns `advds` used to calculate the policy loss (lines 19–21). Then, we update the policy network parameters using the loss (line 22).

Code 2.6 REINFORCE implementation: training method

```

1  # slm_lab/agent/algorithm/reinforce.py
2
3  class Reinforce(Algorithm):
4      ...

```

```

5
6     @lab_api
7     def sample(self):
8         batch = self.body.memory.sample()
9         batch = util.to_torch_batch(batch, self.net.device,
10             ↪ self.body.memory.is_episodic)
11         return batch
12
13     @lab_api
14     def train(self):
15         ...
16         clock = self.body.env.clock
17         if self.to_train == 1:
18             batch = self.sample()
19             ...
20             pdparams = self.calc_pdparam_batch(batch)
21             advs = self.calc_ret_advs(batch)
22             loss = self.calc_policy_loss(batch, pdparams, advs)
23             self.net.train_step(loss, self.optim, self.lr_scheduler,
24                 ↪ clock=clock, global_net=self.global_net)
25             # reset
26             self.to_train = 0
27             return loss.item()
28         else:
29             return np.nan

```

2.6.6 On-Policy Replay Memory

This section looks at a memory class which implements on-policy sampling. We can see on line 17 of Code 2.6 that a memory object is called to produce trajectories for training. Since REINFORCE is an *on-policy* algorithm, trajectories sampled by the algorithm should be stored in a `Memory` class for learning, then flushed after each training step.

Readers uninterested in the details of memory classes can skip this section without loss of understanding of the REINFORCE algorithm. It is sufficient to know what information is stored in memory and how it is used in the training loop.

Let's look at the `OnPolicyReplay` class which implements this logic. It contains the API methods:

1. `reset` clears and resets the memory class variables.
2. `update` adds an experience to the memory.
3. `sample` samples a batch of data for training.

Memory Initialization and Reset `__init__` initializes the class variables, including the storage keys on line 15. It then calls `reset` to construct empty data structures.

`reset` in Code 2.7 is used to clear the memory after each training step. This is specific to an on-policy memory because trajectories cannot be reused for later training.

The memory class can store trajectories from multiple episodes in the attributes initialized in lines 21–22. Individual episodes are constructed by storing experiences in the current episode data dictionary `self.cur_epi_data`, which is reset on line 23.

Code 2.7 OnPolicyReplay: reset

```

1  # slm_lab/agent/memory/onpolicy.py
2
3  class OnPolicyReplay(Memory):
4      ...
5
6      def __init__(self, memory_spec, body):
7          super().__init__(memory_spec, body)
8          # NOTE for OnPolicy replay, frequency = episode; for other classes
9          ↪ below frequency = frames
10         util.set_attr(self, self.body.agent.agent_spec['algorithm'],
11         ↪ ['training_frequency'])
12         # Don't want total experiences reset when memory is
13         self.is_episodic = True
14         self.size = 0 # total experiences stored
15         self.seen_size = 0 # total experiences seen cumulatively
16         # declare what data keys to store
17         self.data_keys = ['states', 'actions', 'rewards', 'next_states',
18         ↪ 'done']
19         self.reset()
20
21     @lab_api
22     def reset(self):
23         '''Resets the memory. Also used to initialize memory vars'''
24         for k in self.data_keys:
25             setattr(self, k, [])
26         self.cur_epi_data = {k: [] for k in self.data_keys}
27         self.most_recent = (None,) * len(self.data_keys)
28         self.size = 0

```

Memory Update The update function serves as an API method to the memory class. Adding an experience to memory is mostly straightforward. The only tricky part is keeping track of the episode boundaries. The steps in Code 2.8 can be broken down as follows:

1. Add the experience to the current episode (lines 14–15).
2. Check if the episode is finished (line 17). This is given by the `done` variable which will be 1 if the episode is finished, 0 otherwise.
3. If the episode is finished, add the entire set of experiences for the episode to the main containers in the memory class (lines 18–19).
4. If the episode is finished, clear the current episode dictionary (line 20) so that the memory class is ready to store the next episode.
5. If the desired number of episodes has been collected, set the `train` flag to 1 in the agent (lines 23–24). This signals that the agent should train this time step.

Code 2.8 OnPolicyReplay: add experience

```

1  # slm_lab/agent/memory/onpolicy.py
2
3  class OnPolicyReplay(Memory):
4      ...
5
6      @lab_api
7      def update(self, state, action, reward, next_state, done):
8          '''Interface method to update memory'''
9          self.add_experience(state, action, reward, next_state, done)
10
11     def add_experience(self, state, action, reward, next_state, done):
12         '''Interface helper method for update() to add experience to memory'''
13         self.most_recent = (state, action, reward, next_state, done)
14         for idx, k in enumerate(self.data_keys):
15             self.cur_epi_data[k].append(self.most_recent[idx])
16         # If episode ended, add to memory and clear cur_epi_data
17         if util.epi_done(done):
18             for k in self.data_keys:
19                 getattr(self, k).append(self.cur_epi_data[k])
20             self.cur_epi_data = {k: [] for k in self.data_keys}
21             # If agent has collected the desired number of episodes, it is
22             ↪ ready to train
23             # length is num of epis due to nested structure
24             if len(self.states) ==
25                 ↪ self.body.agent.algorithm.training_frequency:
26                 self.body.agent.algorithm.to_train = 1
27         # Track memory size and num experiences
28         self.size += 1
29         self.seen_size += 1

```

Memory Sample `sample` in Code 2.9 simply returns all of the completed episodes packaged into a batch dictionary (line 6). Then it resets the memory (line 7) as the stored experiences will no longer be valid once the agent has completed a training step.

Code 2.9 OnPolicyReplay: sample

```

1 # slm_lab/agent/memory/onpolicy.py
2
3 class OnPolicyReplay(Memory):
4
5     def sample(self):
6         batch = {k: getattr(self, k) for k in self.data_keys}
7         self.reset()
8         return batch

```

2.7 Training a REINFORCE Agent

Deep RL algorithms often have many hyperparameters. For example, the network type, architecture, activation functions, optimizer, and learning rate need to be specified. More advanced neural network functionality can involve gradient clipping and learning rate decay schedules. This is only the “deep” part! RL algorithms also involve hyperparameter choices, such as the discount rate, γ , and how frequently to train an agent. To make this more manageable, all of the hyperparameter choices are specified in a JSON file in SLM Lab known as a `spec` (i.e., “specification”); for more details on the `spec` file, see Chapter 11.

Code 2.10 contains an example `spec` file for REINFORCE. The file is also available in SLM Lab at `slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json`.

Code 2.10 A simple REINFORCE cartpole spec file

```

1 # slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json
2
3 {
4     "reinforce_cartpole": {
5         "agent": [{
6             "name": "Reinforce",
7             "algorithm": {
8                 "name": "Reinforce",
9                 "action_pdtype": "default",
10                "action_policy": "default",
11                "center_return": true,
12                "explore_var_spec": null,
13                "gamma": 0.99,
14                "entropy_coef_spec": {

```

```

15         "name": "linear_decay",
16         "start_val": 0.01,
17         "end_val": 0.001,
18         "start_step": 0,
19         "end_step": 20000,
20     },
21     "training_frequency": 1
22 },
23 "memory": {
24     "name": "OnPolicyReplay"
25 },
26 "net": {
27     "type": "MLPNet",
28     "hid_layers": [64],
29     "hid_layers_activation": "selu",
30     "clip_grad_val": null,
31     "loss_spec": {
32         "name": "MSELoss"
33     },
34     "optim_spec": {
35         "name": "Adam",
36         "lr": 0.002
37     },
38     "lr_scheduler_spec": null
39 }
40 }],
41 "env": [{
42     "name": "CartPole-v0",
43     "max_t": null,
44     "max_frame": 100000,
45 }],
46 "body": {
47     "product": "outer",
48     "num": 1
49 },
50 "meta": {
51     "distributed": false,
52     "eval_frequency": 2000,
53     "max_session": 4,
54     "max_trial": 1,
55 },
56 ...
57 }
58 }

```

This may seem like a lot to configure, so it is worth walking through the main components.

- **Algorithm:** The algorithm is REINFORCE (line 8). γ is set on line 13, and we use a baseline for the reinforcing signal by enabling `center_return` on line 11. We add entropy with a linear decay schedule to the loss to encourage exploration (lines 14–20).
- **Network architecture:** Multilayer perceptron with one hidden layer of 64 units and SeLU activation function (lines 27–29).
- **Optimizer:** The optimizer is Adam [68] with a learning rate of 0.002 (lines 34–37). The learning rate is constant throughout because we have specified a null learning rate scheduler (line 38).
- **Training frequency:** Training is episodic because we have selected `OnPolicyReplay` memory (line 24) and the agent is trained at the end of every episode. This is controlled by the `training_frequency` parameter (line 21) since it means the network will be trained every 1 episodes.
- **Environment:** The environment is OpenAI Gym’s `CartPole` [18] (line 42).
- **Training length:** Training consists of 100,000 time steps (line 44).
- **Evaluation:** The agent is evaluated every 2,000 time steps (line 52). During evaluation, four episodes are run, then the mean total rewards are calculated and reported.

To train this REINFORCE agent using SLM Lab, run the commands shown in Code 2.11 in a terminal.

Code 2.11 Training a REINFORCE agent

```
1 conda activate lab
2 python run_lab.py slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json
  ↪ reinforce_cartpole train
```

This will use the specified spec file to run a training `Trial` consisting of four repeated `Sessions`⁷ to obtain an average result, which is then plotted as a trial graph with an error band. Additionally, the trial graph is also plotted with a moving average over a window of 100 evaluation checkpoints to provide a smoothed curve. Both graphs are shown in Figure 2.2.⁸

7. A `Trial` should be run with many `Sessions` using the same spec and different random seeds. This is taken care of automatically in SLM Lab.

8. Total rewards are denoted `mean_returns` in the graphs. For evaluation, `mean_returns` are computed without discounting.

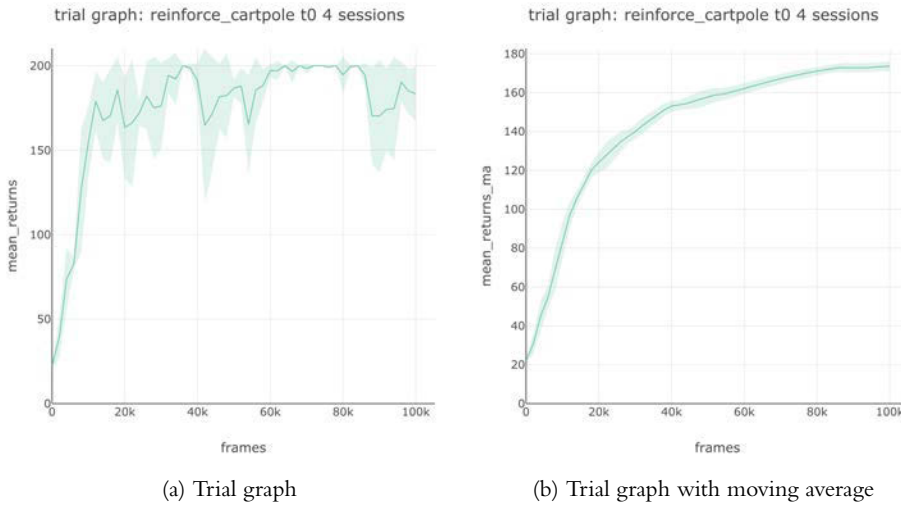


Figure 2.2 REINFORCE trial graphs from SLM Lab, plotted by averaging four repeated sessions. The vertical axis shows the total rewards (denoted `mean_returns`) averaged over eight episodes during checkpoints, and the horizontal axis shows the total training frames. The graph on the right is a smoothed version which plots the moving average with a window of 100 evaluation checkpoints. Note that the maximum total reward for CartPole is 200.

2.8 Experimental Results

In this section, we use the experimentation feature of SLM Lab to study the effects of some components of the algorithm. The first experiment compares the effect of different values of the discount factor γ ; the second shows the improvement from using a baseline in the reinforcing signal.

2.8.1 Experiment: The Effect of Discount Factor γ

The discount factor γ controls the weight of future rewards when calculating the return $R(\tau)$ that is used as reinforcing signal. The higher the discount factor, the more weight is placed on future rewards. The optimal value of γ is problem-specific. For problems where an agent needs to consider the effect of its actions on future rewards over a longer time horizon, γ should be larger.

To study the effects of the discount factor, we can run the REINFORCE algorithm and search over different values of γ . To do this, modify the spec file from Code 2.10 by adding a search spec for γ . This is shown in Code 2.12, where line 15 specifies a grid search over the `gamma` hyperparameter. The full spec file is already available in SLM Lab at `slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json`.

Code 2.12 REINFORCE spec file with search spec for different gamma values

```

1  # slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json
2
3  {
4      "reinforce_cartpole": {
5          ...
6          "meta": {
7              "distributed": false,
8              "eval_frequency": 2000,
9              "max_session": 4,
10             "max_trial": 1,
11         },
12         "search": {
13             "agent": [{
14                 "algorithm": {
15                     "gamma_grid_search": [0.1, 0.5, 0.7, 0.8, 0.90, 0.99, 0.999]
16                 }
17             }]
18         }
19     }
20 }

```

To run the experiment in SLM Lab, use the commands shown in Code 2.13. This is not very different from training an agent—we simply reuse the same spec file and substitute the mode `train` with `search`.

Code 2.13 Run an experiment to search over different gamma values as defined in the spec file.

```

1  conda activate lab
2  python run_lab.py slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json
   ↪ reinforce_cartpole search

```

This will run an `Experiment` which spawns multiple `Trials`, each with a different value of `gamma` substituted into the original REINFORCE spec. Each `Trial` runs four repeated `Sessions` to obtain an average along with an error band. This is used to plot a multitrial graph. A smoothed plot with moving average over 100 evaluation checkpoints is also provided for easier comparison. These are shown in Figure 2.3.

Figure 2.3 shows that γ values above 0.90 perform better, with $\gamma = 0.999$ from trial 6 giving the best result. When γ is too low, the algorithm fails to learn a policy that solves the problem, and the learning curve stays flat.

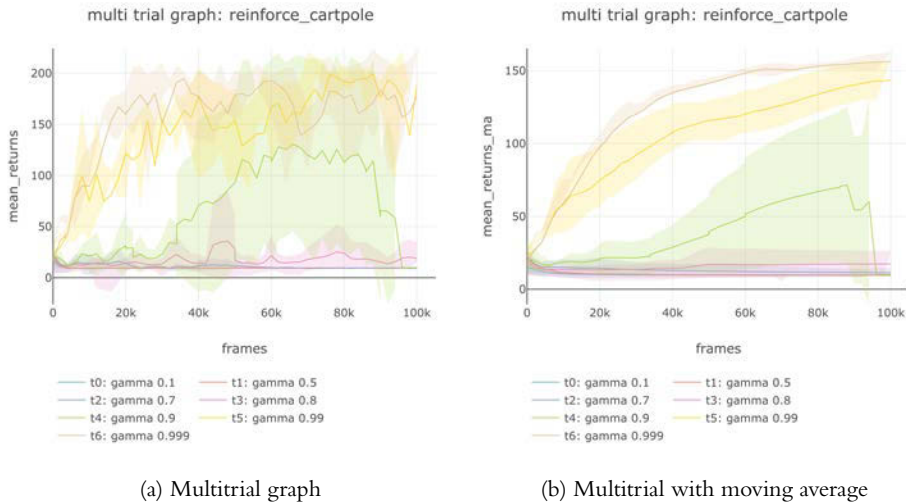


Figure 2.3 The effect of setting the discount factor γ to different values. For CartPole, a lower γ performs worse, and $\gamma = 0.999$ from trial 6 performs best. In general, different problems may have different optimal values for γ .

2.8.2 Experiment: The Effect of Baseline

We learned in this chapter that using a baseline can help reduce the variance of the Monte Carlo policy gradient estimate. Let's run an experiment to compare the performance of REINFORCE with and without a baseline. The baseline can be toggled on or off with the algorithm spec hyperparameter `center_return`.

To run an experiment, we copy the original REINFORCE spec and rename it to `reinforce_baseline_cartpole`. This spec is shown partly in Code 2.14. Then, add a search spec to perform a grid search by toggling `center_return` on and off (line 15). The spec file is also available in SLM Lab at `slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json`.

Code 2.14 REINFORCE spec file with search spec for toggling baseline (`center_return`) on and off

```
1 # slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json
2
3 {
4   "reinforce_baseline_cartpole": {
5     ...
6     "meta": {
7       "distributed": false,
8       "eval_frequency": 2000,
9       "max_session": 4,
```

```

10     "max_trial": 1,
11 },
12 "search": {
13     "agent": [{
14         "algorithm": {
15             "center_return__grid_search": [true, false]
16         }
17     }]
18 }
19 }
20 }

```

To run the experiment in SLM Lab, use the commands shown in Code 2.15.

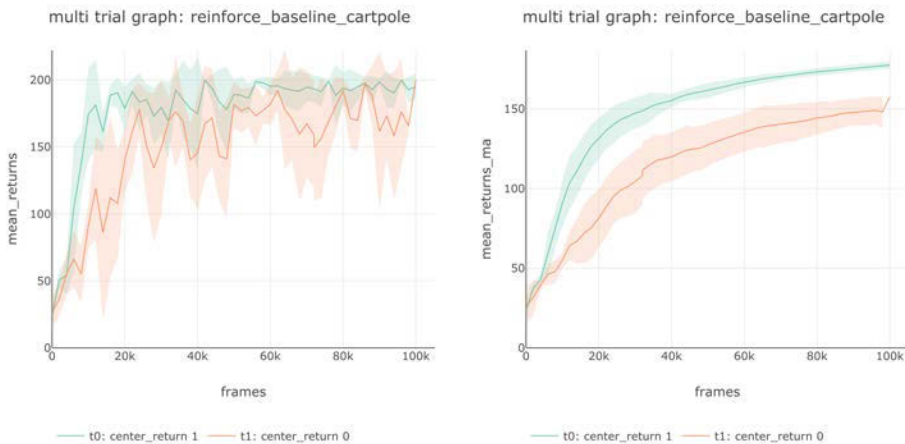
Code 2.15 Run an experiment to compare performance with and without baseline as defined in the spec file.

```

1 conda activate lab
2 python run_lab.py slm_lab/spec/benchmark/reinforce/reinforce_cartpole.json
  ↪ reinforce_baseline_cartpole search

```

This will run an Experiment with two Trials with four Sessions each. The multitrial graph and its moving average over 100 evaluation checkpoints are shown in Figure 2.4.



(a) Multitrial graph

(b) Multitrial graph with moving average

Figure 2.4 These graphs show that when a baseline is used to reduce the variance of policy gradient estimates, performance is improved.

As expected, REINFORCE with a baseline outperforms its variant without a baseline by reducing the variance of policy gradient estimates. Figure 2.4 shows that the baseline variant from trial 0 learns faster and obtains higher total rewards.

This section was a minimal introduction to show how we can run specific experiments using the experimentation feature in SLM Lab. Chapter 11 goes into more detail on experiment design with SLM Lab. We will only need to understand this feature with more advanced use cases, and it is not necessary for understanding the algorithms discussed in this book.

2.9 Summary

This chapter introduced REINFORCE, a policy gradient algorithm. The key idea is that the parameters of a policy network are adjusted to maximize the objective, which is the expected return $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$ for an agent.

We introduced and derived the policy gradient. It is an elegant expression which allows us to adjust the policy action probabilities to encourage good actions and discourage bad ones. In REINFORCE, the policy gradient is estimated using Monte Carlo sampling. This estimate can have high variance, and one common method to reduce it is by introducing a baseline.

The algorithm can be implemented quite simply as shown in Code 2.1. We also discussed its implementation in SLM Lab as an introduction to the algorithm API used throughout this book.

2.10 Further Reading

- “Sep 6: Policy gradients introduction, Lecture 4,” *CS 294: Deep Reinforcement Learning, Fall 2017*, Levine [74].
- “Lecture 14: Reinforcement Learning,” *CS231n: Convolutional Neural Networks for Visual Recognition, Spring 2017* [39].
- “The Beginning of the Monte Carlo Method,” Metropolis, 1987. [85]

2.11 History

Monte Carlo was popularized by Stanislaw Ulam who was working at the Los Alamos research lab in the 1940s. The name “Monte Carlo” does not carry any special connotations; it is simply a memorable alternative to “random estimation.” Still, it does have an entertaining origin. It was suggested by Nicholas Metropolis, a physicist and computer designer, responsible for the wryly named MANIAC computer [5]. Metropolis heard about Ulam’s uncle who borrowed money from relatives because he “just had to go to Monte Carlo.” After that, the name just seemed obvious [85].

During the same period, the ENIAC computer had been developed at the University of Pennsylvania in Philadelphia. A vast machine, consisting of more than 18,000 vacuum tubes, it was one of the earliest general-purpose electronic computers. Ulam was impressed by the computational power and flexibility of the ENIAC, and contemplated that it would be an appropriate tool for the many tedious calculations needed to make use of statistical techniques in function evaluation. John von Neumann, a brilliant polymath and Los Alamos colleague, immediately saw the value of the Monte Carlo approach and in 1947 outlined the technique in a note to the head of the Theoretical Division at Los Alamos [85]. For more information on the emergence of Monte Carlo methods, see “The Beginning of the Monte Carlo Method,” N. Metropolis, *Los Alamos Science* Special Issue, 1987.